

Gabriel Luz · [Follow publication](#)

Terraform

+

Infrastructure As Code

+

O Que É Terraform

+

AWS

+

Terraform 101: Uma breve introdução a Infra as Code

9 min read · Jul 25, 2021



Gabriel Luz

Follow



Share



Introdução — Infra as Code

Olá! Bem vindos a mais um post no blog. Dessa vez vamos falar de um assunto que ganhou muita notoriedade nos últimos anos devido a seus diversos benefícios, Infra as Code (IaC). Esse termo se tornou tão popular especialmente na área de DevOps e já virou regra de boa prática nas empresas que fazem uso dos mais diversos provedores de cloud disponíveis no mercado.

Podemos definir IaC como um método que usa arquivos de definição (em geral arquivos de configuração padronizados, com o json e yaml) para gerenciar e provisionar infraestruturas de TI. Com isso, podemos entender essa prática como uma abordagem de engenharia de software aplicada para operações.

IaC é um processo que trás diversos benefícios tais como a agilidade por conta da capacidade de automação no provisionamento de uma infraestrutura, permitindo a configuração de recursos usando apenas arquivos de definição. Outro grande benefício é a consistência, devido a etapas bem estabelecidas oriundos da IaC, falhas e discrepâncias que seriam criadas por um processo manual são praticamente eliminadas. Segurança também é um fator importante nessa lista, pois as ferramentas de IaC permitem a rápida localização e correção de erros e a solução de problemas de forma automatizada. Com isso, temos que a prática de Infra as Code possui os mesmos princípios seguidos na cultura DevOps no quesito da velocidade e consistência do ciclo de vida de entregas.

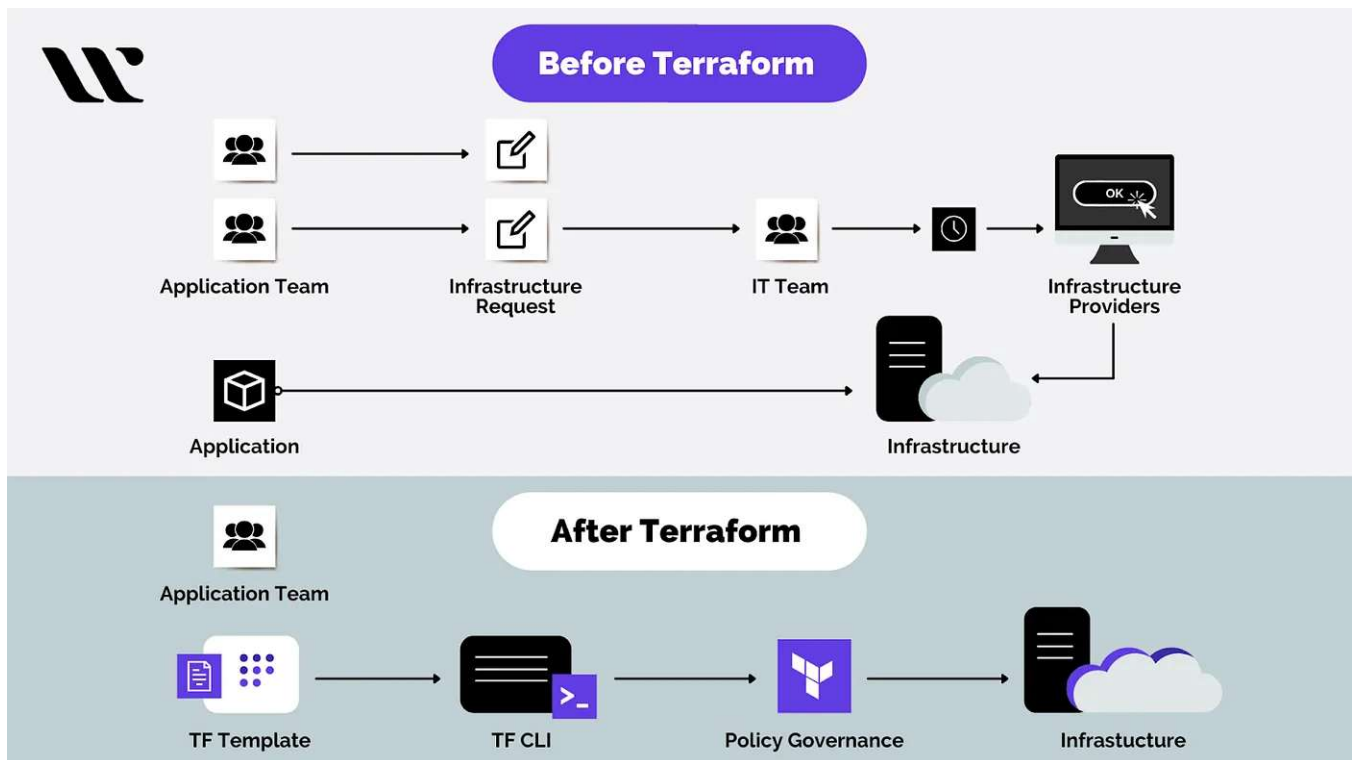
Conceitos de Terraform

Segundo a documentação oficial da Hashicorp, empresa criadora do Terraform, podemos definir a ferramenta da seguinte forma:

“Terraform é uma ferramenta de infraestrutura como código (IaC) que permite criar e alterar versões de infraestrutura com segurança e eficiência. Isso inclui componentes de baixo nível, como instâncias de computação, armazenamento e rede, bem como componentes de alto nível, como entradas de DNS, recursos de SaaS, et.”

É válido mencionar que existem outras ferramentas de IaC muito interessantes, como o CloudFormation da AWS, entretanto, Terraform possui diversos benefícios e vantagens, como os listados abaixo:

- Faz orquestração, não apenas gerenciamento de configuração.
- Suporta vários provedores, como AWS, Azure, Oracle, GCP e muitos mais.
- Fornece infraestrutura imutável onde a configuração muda suavemente.
- Usa linguagem de fácil compreensão, HCL (linguagem de configuração HashiCorp).



Operação do Terraform. Fonte: <https://www.whizlabs.com/blog/what-is-terraform-terraform-tutorial/>

O ciclo de vida de projetos no Terraform pode ser ilustrado com o fluxograma abaixo, contendo o nome da etapa bem como o comando associado:



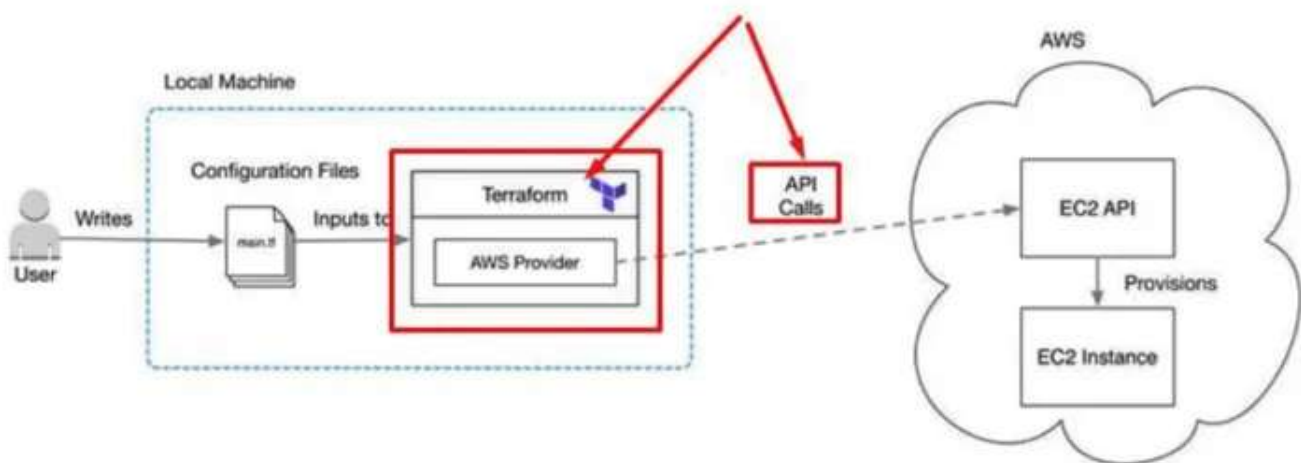
Fonte: <https://k21academy.com/terraform-iac/terraform-beginners-guide/>

1. **terraform init:** inicializa o ambiente Terraform (local). Normalmente é executado apenas uma vez por sessão.
2. **terraform plan:** compara o estado do Terraform com o estado como está na cloud, constrói e exibe um plano de execução.
3. **terraform apply:** executa o plano.
4. **terraform destroy:** exclui todos os recursos controlados por este ambiente específico do terraform.

No contexto de Terraform, existem diversos conceitos importantes para o uso da ferramenta, abaixo estão os principais deles:

1. **Variáveis:** o Terraform possui variáveis de entrada e saída, seguindo o padrão chave-valor. Variáveis de entrada são usadas como parâmetros para personalizar deploys. Variáveis de saída, por sua vez, são valores de retorno de um módulo que podem ser usados por outras configurações.

2. **Provedor:** os usuários do Terraform provisionam sua infraestrutura nos principais provedores de nuvem, como AWS, Azure, GCP e outros. Um provedor é um plug-in que interage com as várias APIs necessárias para criar, atualizar e excluir vários recursos. O Terraform possui diversos provedores para diferentes tecnologias e cada um deles dá ao usuário do Terraform acesso aos seus recursos.



Fonte: <https://k21academy.com/terraform-iac/terraform-beginners-guide/>

3. **Módulo:** qualquer conjunto de arquivos de configuração do Terraform em uma pasta pode ser considerado um módulo. Cada configuração do Terraform tem pelo menos um módulo, conhecido como seu módulo raiz.

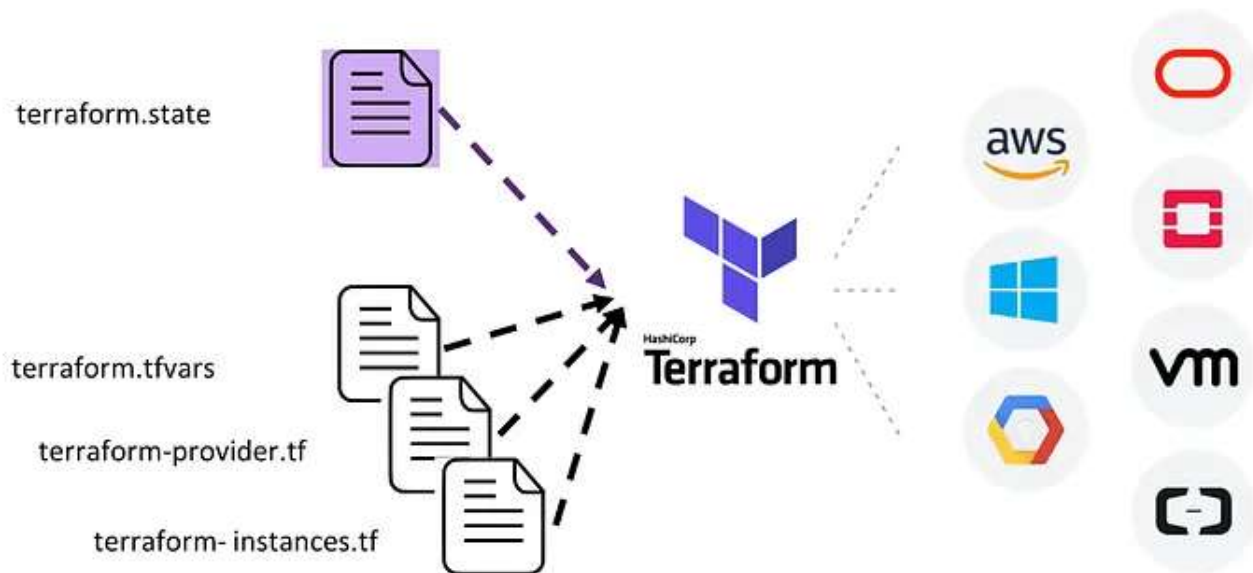
4. **Estado:** o Terraform registra informações sobre qual infraestrutura é criada em um arquivo de estado. Com o arquivo de estado, a ferramenta é capaz de encontrar os recursos que criou anteriormente, para gerenciá-los e atualizá-los de acordo.

5. **Recursos:** provedores de nuvem oferecem vários serviços e eles são chamados de Recursos no Terraform. Os recursos podem ser qualquer coisa, desde instâncias de computação, redes virtuais até componentes de nível superior, como registros DNS. Cada recurso possui seus próprios atributos e parâmetros.

Os arquivos de configuração são essenciais para o uso do Terraform. Eles podem ser definidos como um conjunto de arquivos usados para descrever a infraestrutura e têm as extensões .tf e .tf.json. O Terraform usa um modelo declarativo para definir a

infraestrutura (diferente do imperativo da maioria das linguagens de programação). Uma configuração é composta de um ou mais arquivos em um diretório, como binários do provedor, arquivos de plano e arquivos de estado depois que o Terraform executa a configuração.

1. Arquivo de configuração (arquivos *.tf): aqui, declara-se o provedor e os recursos a serem implantados, juntamente com o tipo de recurso e todas as configurações específicas de cada um.
2. Arquivo de declaração de variável (variables.tf ou variables.tf.json): aqui, declaram-se as variáveis de entrada necessárias para provisionar recursos.
3. Arquivos de definição de variável (terraform.tfvars): aqui, atribui-se valores às variáveis de entrada.
4. Arquivo de estado (terraform.tfstate): um arquivo de estado é criado uma vez após a execução do Terraform. Ele armazena o estado sobre a infraestrutura gerenciada.



Fonte: <https://k21academy.com/terraform-iac/terraform-beginners-guide/>

Hands on com Terraform na AWS

Como explicado anteriormente, uma das grandes vantagens do Terraform é sua capacidade de ser multi-cloud. Para este breve exemplo, irei usar a AWS. De maneira geral, o fluxo de utilização do Terraform segue as etapas abaixo:

- Configure uma conta em qualquer provedor de nuvem (AWS, Azure, GCP etc)
- Instale o Terraform localmente.
- Adicione um provedor — AWS, Azure, GCP etc.
- Escreva os arquivos de configuração
- Inicialize o provedor do Terraform.
- Execute o comando de Plan para visualizar a infraestrutura a ser criada.
- Execute o comando para apply para aplicar as mudanças.
- Execute o comando destroy para eliminar os recursos provisionados.

Para não tornar este tutorial muito grande, vou assumir que o leitor já possui uma conta criada na AWS. Caso ainda não possua, pode seguir as instruções do tutorial deste [link](#). Também é importante ter a AWS CLI, que pode ser instalada seguindo este [tutorial](#).

O passo seguinte é a instalação do Terraform localmente. A ferramenta é compatível com as plataformas Windows, Mac e Linux. Para usuários Mac com o gerenciador de pacotes Homebrew, basta executar o comando “**brew tap hashicorp/tap**” e em seguida “**brew install hashicorp/tap/terraform**”. Para verificar que a instalação foi bem sucedida basta executar o comando “**terraform -help**” ou “**terraform -v**”.

Get Gabriel Luz's stories in your inbox

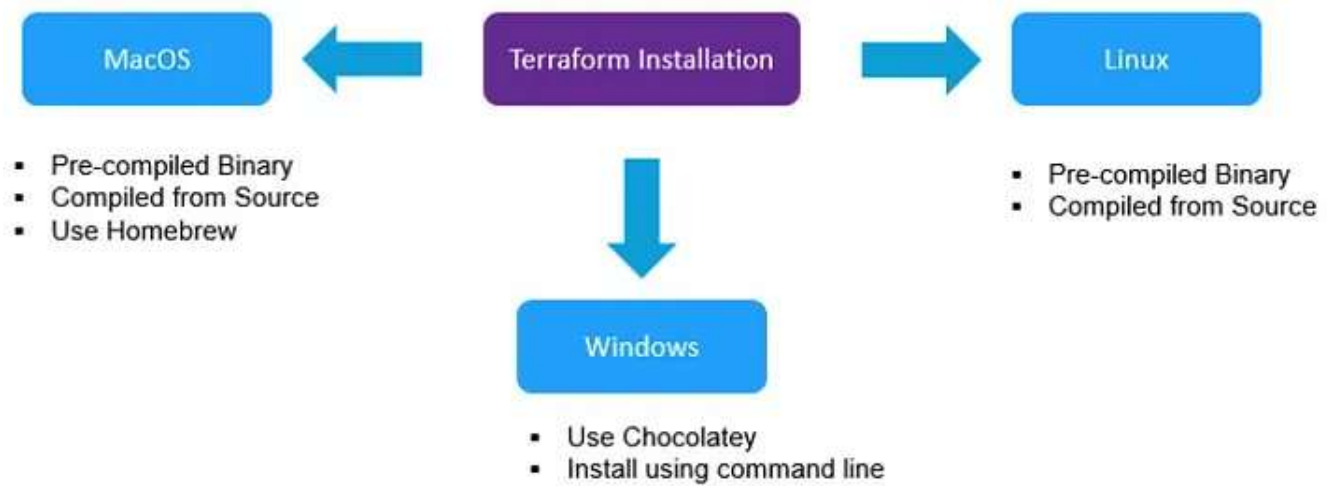
Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Para as demais plataformas e formas de instalação visitar a documentação da Hashcorp através desse [link](#).



Formas de instalação do Terraform. Fonte: <https://k21academy.com/terraform-iac/terraform-beginners-guide/>

O objetivo desse breve exemplo será criar uma instância EC2 na AWS. Para isso, criei um diretório na minha máquina local chamado “terraform_101” e, em seguida, criei o arquivo **awsec2.tf**.

Para usar o Terraform, precisamos realizar a autenticação na AWS com a *access key* e *secret access key*. Existem diferentes formas para fazer isso, mas atenção, não é boa prática setar manualmente as chaves de acesso ou qualquer outro tipo de credencial no código, pois ele será compartilhado em repositórios como o *github* e pessoas mal intencionadas poderiam fazer uso desses dados. Para este exemplo, usarei o comando “**aws configure**” para definir no meu terminal os valores para a *access key*, *secret access key* e região padrão (regiões da AWS).

No arquivo de configuração que foi criado (awsec2.tf) vamos definir a instância da seguinte forma:

Open in app ↗

Sign up

Sign in

Medium

Search



```

aws = {
  source = "hashicorp/aws"
  version = "~> 3.27"
}

required_version = ">= 0.14.9"
  
```

```
provider "aws" {  
  profile = "default"  
  region  = "us-east-1"  
}  
  
resource "aws_instance" "app_server" {  
  ami = "ami-0dc2d3e4c0f9ebd18"  
  instance_type = "t2.micro"  
  tags = {  
    Name = "ExampleAppServerInstance"  
  }  
}
```

Explicando o código, temos que o primeiro bloco (terraform) contém configurações dos provedores necessários para a infraestrutura. Para cada provedor, o atributo *source* define um nome de *host* opcional, um *namespace* e o tipo de provedor. O Terraform instala os provedores do *Terraform Registry* por padrão. Nesta configuração de exemplo, a origem do provedor AWS é definida como *hashicorp/aws*, que é a abreviação de *registry.terraform.io/hashicorp/aws*.

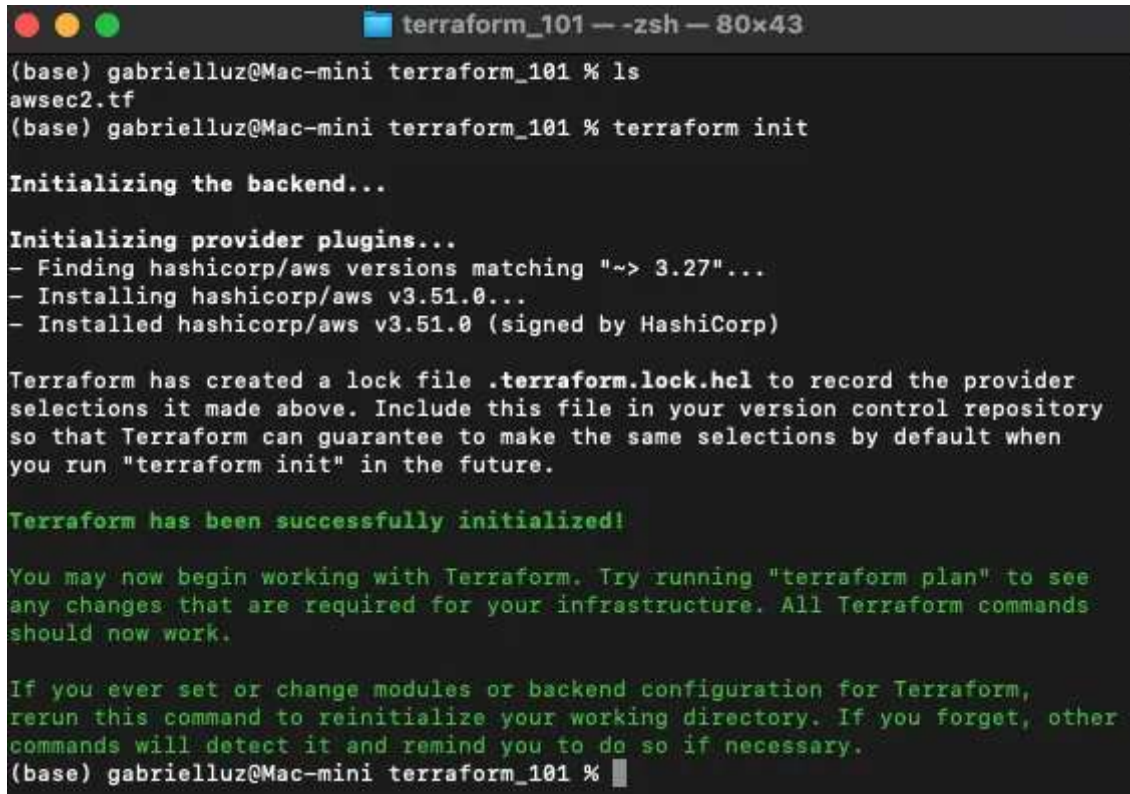
Também é possível definir de versão para cada provedor. O atributo é opcional, mas é recomendado usá-lo para prevenir que o Terraform instale uma versão que não funcione com o estado atual da configuração. Se não especificada, o Terraform irá usar a versão mais recente durante.

O bloco de provedor configura a cloud escolhida, neste caso a AWS. O atributo *profile* se refere às credenciais AWS armazenadas na AWS CLI. O bloco de *resource* define os componentes da infraestrutura. Note que existem duas strings antes do bloco: o tipo de recurso e o seu nome. Neste exemplo, o tipo de recurso é *aws_instance* e o nome é *app_server*. O prefixo do tipo mapeia para o nome do provedor (AWS). Juntos, o tipo de recurso e o nome do recurso formam uma ID exclusiva para o recurso, no caso, o ID da instância EC2 será *aws_instance.app_server*.

Por último, em relação à configuração dos recursos em si, os argumentos podem incluir propriedades como tamanhos de máquina, nomes de imagem de disco ou IDs de VPC. Cada provedor de referência documenta os argumentos obrigatórios e opcionais para cada recurso. Para uma instância EC2, a configuração que usamos

define o AMI ID para uma imagem Ubuntu e o tipo de instância para t2.micro, que faz parte do nível gratuito da AWS. Também definimos uma tag para a instância.

O próximo passo é inicializar o diretório onde o arquivo de configuração está armazenado com o comando “**terraform init**”.

A terminal window titled 'terraform_101 — zsh — 80x43' shows the execution of 'terraform init'. The output indicates that the AWS provider is being installed (version 3.51.0) and a lock file '.terraform.lock.hcl' is created. The message 'Terraform has been successfully initialized!' is displayed in green, followed by instructions on how to use 'terraform plan' and 'terraform init' again if the configuration changes.

```
(base) gabrielluz@Mac-mini terraform_101 % ls
awsec2.tf
(base) gabrielluz@Mac-mini terraform_101 % terraform init

Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 3.27"...
- Installing hashicorp/aws v3.51.0...
- Installed hashicorp/aws v3.51.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
(base) gabrielluz@Mac-mini terraform_101 %
```

Note que o Terraform instala o provedor AWS em um subdiretório oculto no diretório atual, denominado `.terraform`. O comando **terraform init** imprime qual versão do provedor foi instalada e também cria um arquivo de bloqueio denominado `.terraform.lock.hcl`, que especifica as versões exatas do provedor usadas. O próximo passo é executar o comando “**terraform plan**”, que irá mostrar o plano de execução da infraestrutura.

```

terraform_101 — zsh — 80x58
(base) gabrielluz@Mac-mini terraform_101 % terraform plan

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.app_server will be created
+ resource "aws_instance" "app_server" {
+   ami                        = "ami-830c94e3"
+   arn                       = (known after apply)
+   associate_public_ip_address = (known after apply)
+   availability_zone         = (known after apply)
+   cpu_core_count            = (known after apply)
+   cpu_threads_per_core       = (known after apply)
+   disable_api_termination    = (known after apply)
+   ebs_optimized              = (known after apply)
+   get_password_data          = false
+   host_id                   = (known after apply)
+   id                        = (known after apply)
+   instance_initiated_shutdown_behavior = (known after apply)
+   instance_state             = (known after apply)
+   instance_type              = "t2.micro"
+   ipv6_address_count          = (known after apply)
+   ipv6_addresses             = (known after apply)
+   key_name                   = (known after apply)
+   monitoring                 = (known after apply)
+   outpost_arn                = (known after apply)
+   password_data              = (known after apply)
+   placement_group            = (known after apply)
+   primary_network_interface_id = (known after apply)
+   private_dns                = (known after apply)
+   private_ip                 = (known after apply)
+   public_dns                 = (known after apply)
+   public_ip                  = (known after apply)
+   secondary_private_ips       = (known after apply)
+   security_groups             = (known after apply)
+   source_dest_check          = true
+   subnet_id                  = (known after apply)
+   tags                       = {
+     "Name" = "ExampleAppServerInstance"
+   }
+   tags_all                   = {
+     "Name" = "ExampleAppServerInstance"
+   }
+   tenancy                    = (known after apply)
+   user_data                  = (known after apply)
+   user_data_base64           = (known after apply)
+   vpc_security_group_ids     = (known after apply)

+   capacity_reservation_specification {
+     capacity_reservation_preference = (known after apply)

+     capacity_reservation_target {
+       capacity_reservation_id = (known after apply)
+     }
+   }
+ }

```

No output desse comando é possível visualizar o que o Terraform irá criar. Em seguida, o comando “**terraform apply**” executará o arquivo de configuração e iniciará uma instância na AWS. Esse comando irá pedir uma confirmação para a sua execução, basta digitar “yes”.

```
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.app_server: Creating...
aws_instance.app_server: Still creating... [10s elapsed]
aws_instance.app_server: Still creating... [20s elapsed]
aws_instance.app_server: Still creating... [30s elapsed]
aws_instance.app_server: Still creating... [40s elapsed]
aws_instance.app_server: Creation complete after 41s [id=i-0b6535605c4fb8a07]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
(base) gabrielluz@Mac-mini terraform_101 %
```

Após a execução do comando de *apply*, podemos conferir no console da AWS a instância criada.

The screenshot displays the AWS Management Console interface. At the top, the 'Instâncias (1/1)' section shows a table with one instance, 'ExampleAppServerInstance', in the 'running' state. Below this, the detailed view for instance 'i-0b6535605c4fb8a07' is shown, including its ID, state, type, and various network and IAM details.

Nome	ID de instância	Estado da inst...	Tipo de Inst...	Verificação de s...	Status do al...
ExampleAppServerInstance	i-0b6535605c4fb8a07	Executando	t2.micro	2/2 verificações a...	Sem alar...

Instância: i-0b6535605c4fb8a07 (ExampleAppServerInstance)		
Resumo da instância		
ID de instância	Endereço IPv4 público	
i-0b6535605c4fb8a07 (ExampleAppServerInstance)	3.210.203.228 endereço aberto	
Estado da instância	DNS IPv4 público	
Executando	ec2-3-210-203-228.compute-1.amazonaws.com endereço aberto	
Tipo de instância	Endereços IP elásticos	
t2.micro	-	
Descoberta do AWS Compute Optimizer	Função do IAM	
Opte por participar do AWS Compute Optimizer para obter recomendações. Saiba mais	-	
Detalhes da instância		
Plataforma	ID da AMI	Monitoramento

Por último, iremos executar o comando “**terraform destroy**” para destruir o que foi criado e assim não gerar custo após o fim do período de avaliação gratuito da AWS.

```

Plan: 0 to add, 0 to change, 1 to destroy.

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_instance.app_server: Destroying... [id=i-0b6535605c4fb8a07]
aws_instance.app_server: Still destroying... [id=i-0b6535605c4fb8a07, 10s elapse
d]
aws_instance.app_server: Still destroying... [id=i-0b6535605c4fb8a07, 20s elapse
d]
aws_instance.app_server: Still destroying... [id=i-0b6535605c4fb8a07, 30s elapse
d]
aws_instance.app_server: Destruction complete after 34s

Destroy complete! Resources: 1 destroyed.
(base) gabrielluz@Mac-mini terraform_101 %

```

Conferindo no terminal mais uma vez, podemos nos certificar de que a instância foi destruída:



Conclusão

Muito bom! Agora temos um conhecimento básico de como usar o Terraform. Mas estamos apenas arranhando a superfície dessa ferramenta incrível. No futuro, publicarei mais postagens de blog relacionadas ao Terraform. Feedbacks são sempre bem vindos! 😊

Referências:

- <https://www.darede.com.br/o-que-e-infrastructure-as-code-iac/>
- <https://www.terraform.io/intro/index.html>
- <https://k21academy.com/terraform-iac/terraform-beginners-guide/>
- <https://geekflare.com/terraform-for-beginners/>

- <https://towardsdatascience.com/terraform-101-d51437a3170>

Terraform

+

Infrastructure As Code

+

O Que É Terraform

+

AWS

+



Follow

Written by Gabriel Luz

188 followers · 82 following

Estudante de engenharia eletrônica, aspirante a cientista de dados e apaixonado por tecnologia.



Follow

Published in Gabriel Luz

54 followers · Last published Mar 25, 2023

Blog sobre projetos pessoais nas áreas de ciência e engenharia de dados